

Real-Time Telemetry Graph Implementation Spec

Target Framework: Avalonia UI (C# / .NET 10) | **Architecture:** MVVM (CommunityToolkit.Mvvm)

Author: Frontend Team

1. Overview & Objective

The goal of this initiative is to integrate an active, live-style moving graph on the main dashboard of the Auditor Desktop App. The graph serves to:

- Make the application dashboard visually dynamic, modern, and engaging.
- Provide real-time visibility into system health, API response times, and multi-user network activity.
- Maintain top performance without blocking or lagging the Avalonia UI main thread.

2. Selected Tech Stack & Libraries

All libraries selected for this feature are open-source and 100% free for commercial use (MIT License):

Component	Technology	Purpose
UI Framework	Avalonia UI	Cross-platform desktop application framework.
Charting Engine	LiveChartsCore.SkiaSharpView.Avalonia	Hardware-accelerated (SkiaSharp) animated line/area charting.
Data Streaming	Microsoft.AspNetCore.SignalR.Client	Real-time WebSocket connection to push telemetry ticks from API.
MVVM Toolkit	CommunityToolkit.Mvvm	Data binding and reactive view model updates.

3. Metric Selected: API Latency & Network Activity

To provide authentic multi-user system data, the graph will track Global API Response Latency (in milliseconds) combined with Active Request Volume.

Why this metric? Unlike local system memory (which produces sharp sawtooth patterns), API latency fluctuates naturally between ~15ms and ~45ms, creating a smooth, stock-market ticker visual while giving users real feedback on server responsiveness.

4. Implementation Steps & Execution Order

Developers should execute the implementation in the exact order detailed below:

- Step 1 → Package Installation & Setup
- Step 2 → Implement Dashboard ViewModel & Sliding Window
- Step 3 → Build & Style Avalonia XAML View
- Step 4 → Backend Telemetry Hookup (SignalR Integration)

Step 1: Package Installation & Setup

Add the required charting and messaging NuGet packages to the desktop client project:

```
dotnet add package LiveChartsCore.SkiaSharpView.Avalonia
dotnet add package Microsoft.AspNetCore.SignalR.Client
```

Step 2: ViewModel Implementation (DashboardViewModel.cs)

The ViewModel manages a fixed sliding window of 30 visible data points (MaxVisiblePoints). New data points enter on the right, while the oldest point is removed from the left to keep the graph scrolling smoothly.

```
using System;
using System.Collections.ObjectModel;
using System.Threading;
using System.Threading.Tasks;
using CommunityToolkit.Mvvm.ComponentModel;
using LiveChartsCore;
using LiveChartsCore.Defaults;
using LiveChartsCore.SkiaSharpView;
using LiveChartsCore.SkiaSharpView.Painting;
using Microsoft.AspNetCore.SignalR.Client;
using SkiaSharp;

namespace AuditApp.ViewModels;

public partial class DashboardViewModel : ObservableObject
{
    private readonly Random _random = new();
    private HubConnection? _hubConnection;
    private const int MaxVisiblePoints = 30;

    public ObservableCollection<ISeries> TelemetrySeries { get; set; }
    public ObservableCollection<ObservableValue> RealtimePoints { get; set; } = new();

    public DashboardViewModel()
    {
        // Populate initial baseline values
        for (int i = 0; i < MaxVisiblePoints; i++)
        {
            RealtimePoints.Add(new ObservableValue(25));
        }

        // Configure smooth financial spline area styling
        TelemetrySeries = new ObservableCollection<ISeries>
        {
            new LineSeries<ObservableValue>
            {
                Values = RealtimePoints,
                Fill = new SolidColorPaint(SKColors.Emerald.WithAlpha(35)),
                Stroke = new SolidColorPaint(SKColors.Emerald, 2f),
                GeometrySize = 0, // Hides point dots for a clean line
            }
        }
    }
}
```

```

        LineSmoothness = 0.55
    }
};

// Start mock stream for frontend testing
_ = StartMockDataStreamAsync();
}

private async Task StartMockDataStreamAsync()
{
    using var timer = new PeriodicTimer(TimeSpan.FromMilliseconds(400));

    while (await timer.WaitForNextTickAsync())
    {
        double simulatedLatency = 25 + (_random.NextDouble() * 18 - 8);
        AddPointToChart(simulatedLatency);
    }
}

public void AddPointToChart(double value)
{
    Avalonia.Threading.Dispatcher.UIThread.Post(() =>
    {
        RealtimePoints.Add(new ObservableValue(Math.Round(value, 1)));

        if (RealtimePoints.Count > MaxVisiblePoints)
        {
            RealtimePoints.RemoveAt(0);
        }
    });
}
}

```

Step 3: XAML Chart View (DashboardView.axaml)

Embed the CartesianChart control into the dashboard UI. Both axes are hidden to mirror financial stock widgets.

```

<UserControl x:Class="AuditApp.Views.DashboardView"
    x:DataType="vm:DashboardViewModel"
    xmlns="https://github.com/avaloniaui"
    xmlns:lvc="using:LiveChartsCore.SkiaSharpView.Avalonia"
    xmlns:vm="using:AuditApp.ViewModels"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">

    <Border Background="#1E1E2E" CornerRadius="12" Margin="10" Padding="16">

        <Grid RowDefinitions="Auto, *">
            <!-- Header Status -->
            <Grid ColumnDefinitions="*, Auto" Grid.Row="0" Margin="0,0,0,10">
                <StackPanel Orientation="Horizontal" Spacing="8">
                    <Ellipse Fill="#10B981" Height="8" VerticalAlignment="Center" Width="8"/>
                    <TextBlock FontSize="12" FontWeight="SemiBold" Foreground="#A6ADC8"
                        Text="SYSTEM LATENCY & NETWORK ACTIVITY"/>
                </StackPanel>

                <TextBlock FontSize="10" FontWeight="Bold" Foreground="#10B981"
                    Grid.Column="1" Text="LIVE STREAM"/>
            </Grid>

            <!-- Chart Control -->

```

```

        <lvc:CartesianChart Grid.Row="1"
                           Series="{Binding TelemetrySeries}"
                           TooltipPosition="Hidden"
                           Height="140">
            <lvc:CartesianChart.XAxes>
                <lvc:Axis Visible="False"/>
            </lvc:CartesianChart.XAxes>
            <lvc:CartesianChart.YAxes>
                <lvc:Axis Visible="False" MinLimit="0"/>
            </lvc:CartesianChart.YAxes>
        </lvc:CartesianChart>
    </Grid>
</Border>
</UserControl>

```

Step 4: Connecting to Backend Endpoint (SignalR)

When backend telemetry endpoints go live, replace `StartMockDataStreamAsync()` with the SignalR client connection:

```

private async Task ConnectToBackendStreamAsync()
{
    _hubConnection = new HubConnectionBuilder()
        .WithUrl("https://your-api-domain.com/hubs/telemetry")
        .WithAutomaticReconnect()
        .Build();

    _hubConnection.On<double>("ReceiveTelemetryTick", (latencyMs) =>
    {
        AddPointToChart(latencyMs);
    });

    try
    {
        await _hubConnection.StartAsync();
    }
    catch (Exception)
    {
        // Fallback to mock loop if backend is offline
        _ = StartMockDataStreamAsync();
    }
}

```

5. UI Customization Checklist

- ✅ Set `GeometrySize = 0` to disable individual point markers.
- ✅ Set `LineSmoothness = 0.55` for curved financial splines.
- ✅ Ensure chart points stay bounded (≤ 30) to protect rendering performance.
- ✅ Wrap UI thread updates using `Avalonia.Threading.Dispatcher.UIThread.Post()`.